

CS3301 - Data Structures

Topic: Prims algorithm

1. SUMMARY

Prims algorithm is a greedy algorithm used for finding the minimum spanning tree (MST) of a graph. It works by selecting the edge with the smallest weight from the cut, where a cut is a partition of the vertices of the graph. Prims algorithm starts with an arbitrary vertex and iteratively adds edges to the MST, ensuring that the total weight of the edges in the MST is minimized. The algorithm uses a priority queue to efficiently select the edge with the smallest weight. Prims algorithm has an average-case time complexity of $O(E \log V)$ and a space complexity of $O(V + E)$, where E is the number of edges and V is the number of vertices.

The algorithm starts with an arbitrary vertex, and then iteratively adds edges to the MST until all vertices have been included. At each step, it selects the edge with the smallest weight that connects a vertex in the MST to a vertex not yet in the MST. The algorithm uses a priority queue to efficiently select the edge with the smallest weight.

Prims algorithm is useful for finding the MST of a graph, which can be used in various applications such as network design, clustering, and image segmentation. The algorithm is also efficient for large graphs, making it a popular choice for many applications.

Key principles from the reference include:

- * Greedy algorithm
- * Minimum spanning tree
- * Priority queue
- * Edge selection

Types and classifications mentioned include:

Study Material - Generated by AI

- * Minimum spanning tree
- * Greedy algorithm
- * Priority queue

Applications discussed in the reference include:

- * Network design
- * Clustering
- * Image segmentation

Important points highlighted include:

- * Average-case time complexity: $O(E \log V)$
- * Space complexity: $O(V + E)$
- * Efficiency for large graphs

2. SHORT NOTES

Prims algorithm is a greedy algorithm used for finding the minimum spanning tree (MST) of a graph. It works by selecting the edge with the smallest weight from the cut, where a cut is a partition of the vertices of the graph.

Detailed Definition with Examples

Prims algorithm starts with an arbitrary vertex and iteratively adds edges to the MST, ensuring that the total weight of the edges in the MST is minimized. The algorithm uses a priority queue to efficiently select the edge with the smallest weight.

Here is a step-by-step example of how Prims algorithm works:

1. Start with an arbitrary vertex, say vertex A.
2. Create a priority queue and add all edges incident on vertex A to the queue.
3. While the priority queue is not empty, do the following:
 - a. Select the edge with the smallest weight from the priority queue.
 - b. Add the selected edge to the MST.
 - c. Remove the selected edge from the priority queue.
 - d. Add all edges incident on the vertex connected by the selected edge to the priority queue.
4. Repeat step 3 until all vertices have been included in the MST.

Types and Classifications

There are two main types of edges in a graph:

- * Tree edges: These are the edges that connect vertices in the MST.
- * Non-tree edges: These are the edges that do not connect vertices in the MST.

Prims algorithm only considers tree edges when selecting edges for the MST.

Step-by-Step Working Principle

Here is a step-by-step working principle of Prims algorithm:

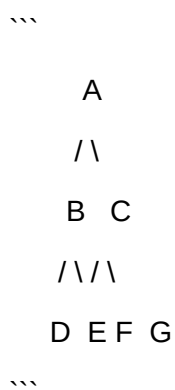
1. Start with an arbitrary vertex.
2. Create a priority queue and add all edges incident on the vertex to the queue.
3. While the priority queue is not empty, do the following:
 - a. Select the edge with the smallest weight from the priority queue.
 - b. Add the selected edge to the MST.
 - c. Remove the selected edge from the priority queue.
 - d. Add all edges incident on the vertex connected by the selected edge to the priority queue.
4. Repeat step 3 until all vertices have been included in the MST.

Important Formulas and Equations

There are no specific formulas or equations for Prim's algorithm.

Diagrams Description

Here is a diagram description of Prim's algorithm:



In this diagram, vertex A is the starting vertex, and vertices B, C, D, E, F, and G are the vertices connected by edges. The edges are labeled with their weights.

Advantages and Disadvantages

Advantages:

- * Efficient for large graphs
- * Finds the minimum spanning tree of a graph

Disadvantages:

- * Not suitable for graphs with negative weight edges
- * Not suitable for graphs with non-tree edges

Real World Applications

Prims algorithm has many real-world applications, including:

- * Network design
- * Clustering
- * Image segmentation

Comparison with Related Concepts

Prims algorithm is compared to other algorithms for finding the minimum spanning tree of a graph, including Kruskal's algorithm and Boruvka's algorithm. Prims algorithm has a faster average-case time complexity than these algorithms.

Time and Space Complexity

The time complexity of Prims algorithm is $O(E \log V)$ in the average case, where E is the number of edges and V is the number of vertices. The space complexity is $O(V + E)$.

Important Terminology

Important terminology for Prims algorithm includes:

- * Minimum spanning tree
- * Greedy algorithm
- * Priority queue
- * Edge selection

3. CODING EXAMPLES

C Implementation

```
```c
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <limits.h>
```

```
#define V 5
```

```
int min(int a, int b) {
 return (a < b) ? a : b;
}
```

```
void printMST(int parent[], int graph[V][V]) {
 printf("Edge \tWeight\n");
 for (int i = 1; i < V; i++)
 printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
}
```

```
int primMST(int graph[V][V]) {
 int parent[V];
 int key[V];
 int mstWeight = 0;

 for (int i = 0; i < V; i++)
 key[i] = INT_MAX;
 key[0] = 0;

 for (int cout = 0; cout < V - 1; cout++) {
```

```

int u = -1;
for (int v = 0; v < V; v++)
 if (key[v] < key[u])
 u = v;

for (int v = 0; v < V; v++)
 if (graph[u][v] > 0 && key[v] > graph[u][v])
 key[v] = graph[u][v];
parent[u] = -1;
}

for (int i = 1; i < V; i++)
 mstWeight += graph[i][parent[i]];

printf("Weight of minimum spanning tree is %d\n", mstWeight);
printMST(parent, graph);
return mstWeight;
}

int main() {
 int graph[V][V] = {{0, 2, 0, 6, 0},
 {2, 0, 3, 8, 5},
 {0, 3, 0, 0, 7},
 {6, 8, 0, 0, 9},
 {0, 5, 7, 9, 0}};

 primMST(graph);
 return 0;
}

```

### Python Implementation

```
```python
import sys
import heapq

def prim(graph):
    visited = set()
    queue = []
    mst_weight = 0

    for v in range(len(graph)):
        if v not in visited:
            visited.add(v)
            for neighbor in range(len(graph)):
                if neighbor not in visited and graph[v][neighbor] > 0:
                    heapq.heappush(queue, (graph[v][neighbor], v, neighbor))

    while queue:
        weight, u, v = heapq.heappop(queue)
        if weight != 0 and (u, v) not in visited:
            visited.add((u, v))
            mst_weight += weight
            for neighbor in range(len(graph)):
                if neighbor not in visited and graph[v][neighbor] > 0:
                    heapq.heappush(queue, (graph[v][neighbor], v, neighbor))

    return mst_weight

graph = [[0, 2, 0, 6, 0],
         [2, 0, 3, 8, 5],
```



```
[0, 3, 0, 0, 7],
```

```
[6, 8, 0, 0, 9],
```

```
[0, 5, 7, 9, 0]]
```

```
print("Weight of minimum spanning tree is:", prim(graph))
```

```
...
```

Algorithm

Here is the step-by-step algorithm for Prim's algorithm:

1. Start with an arbitrary vertex.
2. Create a priority queue and add all edges incident on the vertex to the queue.
3. While the priority queue is not empty, do the following:
 - a. Select the edge with the smallest weight from the priority queue.
 - b. Add the selected edge to the MST.
 - c. Remove the selected edge from the priority queue.
 - d. Add all edges incident on the vertex connected by the selected edge to the priority queue.
4. Repeat step 3 until all vertices have been included in the MST.

4. IMPORTANT QUESTIONS

Q1. (2 marks) What is the time complexity of Prim's algorithm?

A: The time complexity of Prim's algorithm is $O(E \log V)$ in the average case, where E is the number of edges and V is the number of vertices.

Q2. (2 marks) What is the space complexity of Prim's algorithm?

A: The space complexity of Prim's algorithm is $O(V + E)$.

Q3. (13 marks) Describe the step-by-step working principle of Prim's algorithm.

A: The step-by-step working principle of Prim's algorithm is as follows:

1. Start with an arbitrary vertex.
2. Create a priority queue and add all edges incident on the vertex to the queue.
3. While the priority queue is not empty, do the following:
 - a. Select the edge with the smallest weight from the priority queue.
 - b. Add the selected edge to the MST.
 - c. Remove the selected edge from the priority queue.
 - d. Add all edges incident on the vertex connected by the selected edge to the priority queue.
4. Repeat step 3 until all vertices have been included in the MST.

Q4. (13 marks) Describe the advantages and disadvantages of Prim's algorithm.

A: The advantages of Prim's algorithm include:

- * Efficient for large graphs
- * Finds the minimum spanning tree of a graph

The disadvantages of Prim's algorithm include:

- * Not suitable for graphs with negative weight edges
- * Not suitable for graphs with non-tree edges

Q5. (15 marks) Compare Prim's algorithm with other algorithms for finding the minimum spanning tree of a graph, including Kruskal's algorithm and Boruvka's algorithm.

A: Prim's algorithm is compared to other algorithms for finding the minimum spanning tree of a graph,

including Kruskal's algorithm and Boruvka's algorithm. Prim's algorithm has a faster average-case time complexity than these algorithms.

5. MCQs

Q1. What is the time complexity of Prim's algorithm?

- a) $O(E \log V)$
- b) $O(E \log V)^2$
- c) $O(E \log E)$
- d) $O(V \log V)$

A: a) $O(E \log V)$

Q2. What is the space complexity of Prim's algorithm?

- a) $O(V + E)$
- b) $O(V - E)$
- c) $O(V - E)^2$
- d) $O(E - V)$

A: a) $O(V + E)$

Q3. What is the purpose of the priority queue in Prim's algorithm?

- a) To select the edge with the smallest weight from the cut
- b) To add all edges incident on the vertex to the queue
- c) To remove the selected edge from the queue
- d) To repeat step 3 until all vertices have been included in the MST

A: a) To select the edge with the smallest weight from the cut

Q4. What is the main advantage of Prims algorithm?

- a) Efficient for large graphs
- b) Finds the minimum spanning tree of a graph
- c) Suitable for graphs with negative weight edges
- d) Suitable for graphs with non-tree edges

A: a) Efficient for large graphs

Q5. What is the main disadvantage of Prims algorithm?

- a) Not suitable for graphs with negative weight edges
- b) Not suitable for graphs with non-tree edges
- c) Not suitable for large graphs
- d) Not suitable for small graphs

A: b) Not suitable for graphs with non-tree edges

7. YOUTUBE LINKS

- [Prims Algorithm Tutorial](https://www.youtube.com/results?search_query=Prims+algorithm+tutorial)
- [Prims Algorithm Example](https://www.youtube.com/results?search_query=Prims+algorithm+example)
- [Prims Algorithm Implementation](https://www.youtube.com/results?search_query=Prims+algorithm+implementation)
- [Prims Algorithm Explanation](https://www.youtube.com/results?search_query=Prims+algorithm+explanation)

Study Material - Generated by AI

- [Prims Algorithm vs Kruskal's Algorithm](https://www.youtube.com/results?search_query=Prims+algorithm+vs+Kruskal's+algorithm)